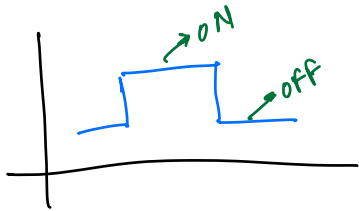


Why TLE occurs?

→ code → gets executed on the platform's servers.

↓
 10^9 clocks cycles/sec

1 cc:-



In general terms:-

1 cc:- 1 instruction.

Server → 10^9 cc/sec → 10^9 instructions/sec

Expected execution time → 1 second.

↓
Our code can have at max 10^9 instructions

```

e.g. for (i=0; i < N; i++) {
    if (i % 2 == 0) {
        print("even");
    }
    else {
        print("odd");
    }
}

```

Annotations:
- 'one time' points to the first iteration of the for loop.
- '++' points to the increment operation in the for loop.
- '++' points to the increment operation in the if statement.
- '++' points to the increment operation in the else statement.
- A bracket on the right side of the if-else block is labeled '++'.

iterations = N

instructions in 1 iteration = 5 instructions.

total instructions = $5N$

Assumption 1.

1 iteration = 10 instructions.

// At max, we can have 10^9 instructions.

= $10^8 \times 10$ instructions. → 1 iteration

// At max, we can have 10^8 iterations.

Assumption 2

1 iteration = 100 instructions.

// At max, we can have 10^9 instructions.

= $10^7 \times 100$ instructions.
→ 1 iteration

// At max, we can have 10^4 iterations.

In general, $[10^7 - 10^8]$ iterations.

What you read?

→ Statement

→ constraints → most people ignore this.
→ uimp

Q1 Equilibrium index.

Constraints:-

size of array
 $1 \leq N \leq 10^5$

$1 \leq arr[i] \leq 10^9$
values in the array.

idea 1:- $T.C = O(N^2)$

At max, $N = 10^5$

iterations = $N^2 = 10^{10}$

↓
This will give TLE.

idea 2:- $pf[9]$.

$T.C = O(N)$

At max, $N = 10^5$

iterations = 10^5 → This will work
No TLE

Q2

$1 \leq N \leq 10^3$

$1 \leq arr[i] \leq 10^4$

→ $O(N^3)$ → 10^9 iterations → It might work

→ $O(N^2)$ → 10^6 iterations → No TLE.

Given a question:-

- 1) Read the question. $\rightarrow$ input / output
- 2) Look at the constraints.
- 3) idea $\rightarrow O(N)$, $O(N^2)$ etc.
- 4) TLE or not
- 5) Edge cases.
- 6) Write code $\rightarrow$ comes at the end.

Q Max contiguous subarray-

$1 \leq N \leq 10^6$ $\rightarrow O(N^2) \rightarrow X$
 $\rightarrow O(N) \checkmark$
 $\rightarrow$ Advance

$$1e6 = 10^6$$

$$1e6 + 7 = 10^6 + 7$$

$$1e9 + 7 = 10^9 + 7$$

$\rightarrow$ exponent with base 10.

Big Prime Number.

$\rightarrow$ Why? $\rightarrow$ Hashing

Quest Given N array elements.

Print the start and end index of all subarrays of size K .

$K \leq N$

arr[12]:
0 1 2 3 4 5 6 7 8 9 10 11
3 4 2 -1 6 7 8 9 3 2 -1 4

$K=3$

s e
[0 2]
[1 3]
[2 4]
[3 5]
[4 6]
[5 7]
⋮
[9 11]
↓
12-3
N=12
K=3

$K=6$

s e
[0 5]
[1 6]
[2 7]
[2 8]
[4 9]
[5 10]
[6 11]
↓
12-6
N=12
K=6

General, N elements, K size

s e
[0 K-1]
[1 K]
[2 K+1]
[3 K+2]
⋮
⋮

[N-K, N-1]

→ $(N-1) - (N-K) + 1$
⇒ $-1 + K + 1$
⇒ K elements.

// Pseudo Code.

```
int s=0, e=K-1; // e < N
while (s <= N-K) {
    print(s + " " + e);
    s++; e++;
}
```

Exact no. of iterations = $N-K+1$ (s goes from $0 \rightarrow N-K$)

T.C = $O(N)$

Note:- Given N array elements, how many subarrays of length $K = N-K+1$

Ques 2 Given N array elements. Find max subarray sum of len = k.

eg. arr[10] :- ⁰-3 ¹4 ²-2 ³5 ⁴3 ⁵-2 ⁶8 ⁷2 ⁸-1 ⁹4

k=5

s	e	sum
[0	4]	7
[1	5]	8
[2	6]	12
[3	7]	16
[4	8]	10
[5	9]	11

ans = 16

ideas:-

→ for every subarray of len k, iterate and get sum.
Return max out of all the sum values.

Pseudo Code:-

ans = ?
 $\begin{cases} 0 \rightarrow -ve \text{ values.} \rightarrow \text{Wrong.} \\ arr[0] \rightarrow \text{invalid ans.} \\ \rightarrow \text{INT-MIN} \end{cases}$

s = 0; e = k-1

while (s ≤ N-k) {

[s, e] is a subarray

sum = 0

for (i = s; i ≤ e; i++) {
 sum += arr[i];
}

ans = max(ans, sum);
 s++, e++;

}
 return ans;

optimise
using
pf[]

exact # of iterations =

$(N-k+1) * k$

iterations.

k=1 N

k=N N

k=N/2 $\left(\frac{N}{2}+1\right) \left(\frac{N}{2}\right)$

⇒ $O(N^2)$

W.C. time complexity.

→ Optimisation using pf[].

1) Construct a pf[] array.

2) for every subarray from [s, e] you get sum using pf[] in $O(1)$

S.C = $O(N)$

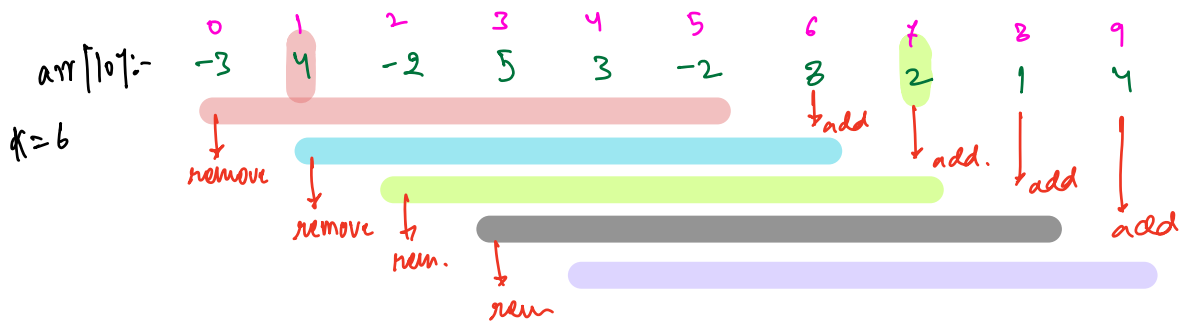
T.C = $O(N + (N-k+1) * O(1)) \Rightarrow O(N)$

1) Without using pf[]

T.C = $O(N)$ and S.C = $O(1)$

2) You cannot modify i/p array.

Break → 10:32 pm.



s e : sum

[0 5] : 5

[1 6] : $5 - arr[0] + arr[6] = 5 - (-3) + 8 = 16$

[2 7] : $16 - arr[1] + arr[7] = 16 - 4 + 2 = 14$

[3 8] :- $14 - arr[2] + arr[8] = 14 - (-2) + 1 = 17$

[4 9] :- $17 - arr[3] + arr[9] = 17 - 5 + 4 = 16$

max = 17

Sliding Window.

Generalise.

N = 0 1 2 3 4 5 ... k-1 k k+1 k+2 ... N-1

s e sum

[0 k-1] :- iterate and get sum. $\rightarrow$ sum

[1 k] :- $sum = sum - arr[s-1] + arr[k]$

[2 k+1] :- $sum = sum - arr[s-1] + arr[k+1]$

[3 k+2] :- $sum = sum - arr[s-1] + arr[k+2]$

$\rightarrow$ We are deleting $arr[s-1]$

$\rightarrow$ We are adding $arr[e]$

} for subarray from [s, e]

// Pseudo Code:-

sum = 0; ans = -

```
for (i = 0; i < K; i++) {  
    sum = sum + arr[i];  $\rightarrow$  first subarray sum.  
}
```

} $\rightarrow$ iterations = K

int s = 1, e = K

ans = sum;

while (s <= N - K) {

```
    [s, e] is a subarray.  
    sum = sum - arr[s-1] + arr[e];  
    ans = max(ans, sum);  
    s++;  
    e++;  
}
```

} $\rightarrow$ iterations = N - K

return ans;

total iterations = ~~K * N~~
= N

q.c = $O(N)$

s.c = $O(1)$

Spiral Matrix $\rightarrow$ Problem Solving.

Maximum Swap $\rightarrow$